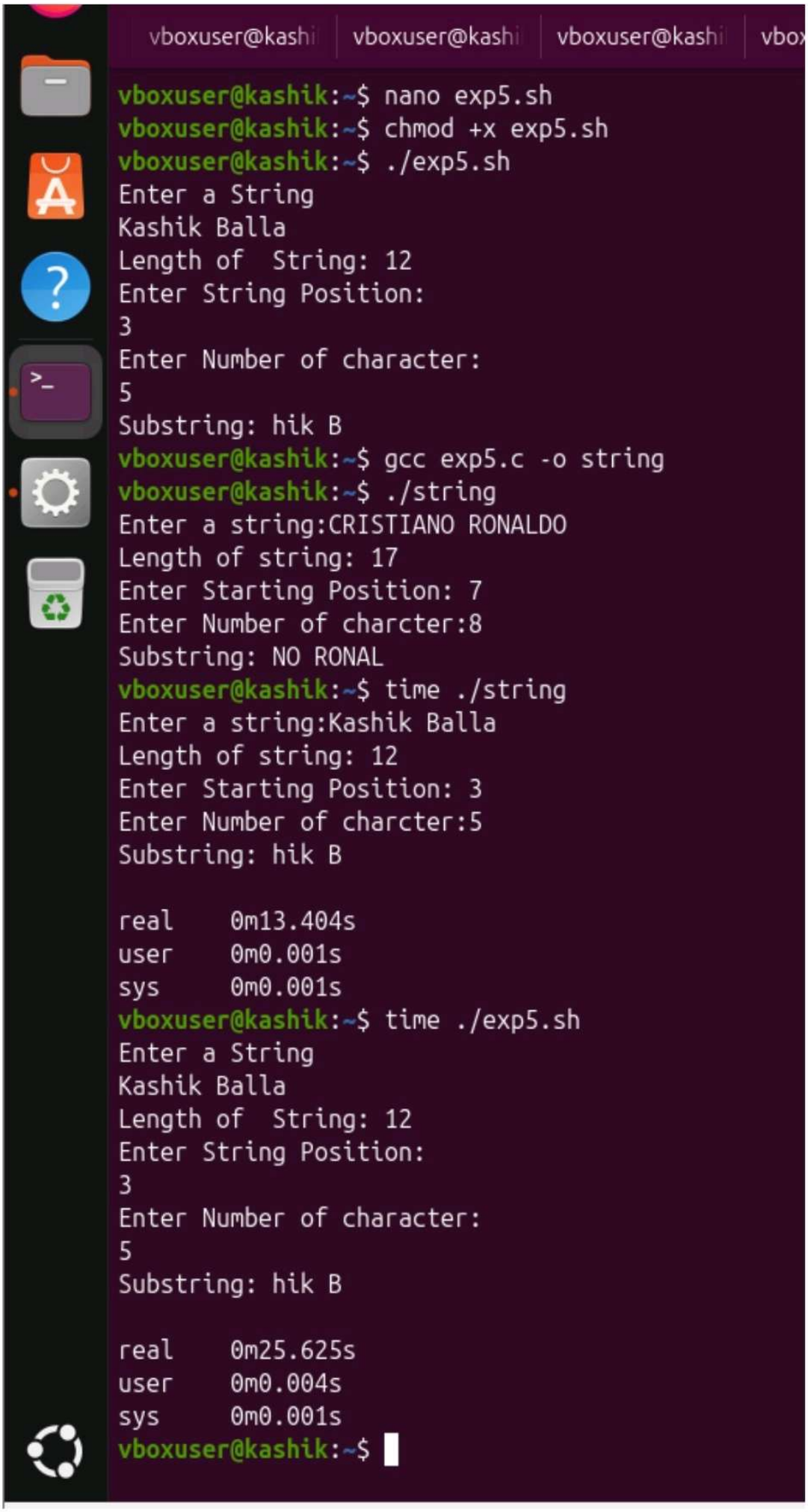
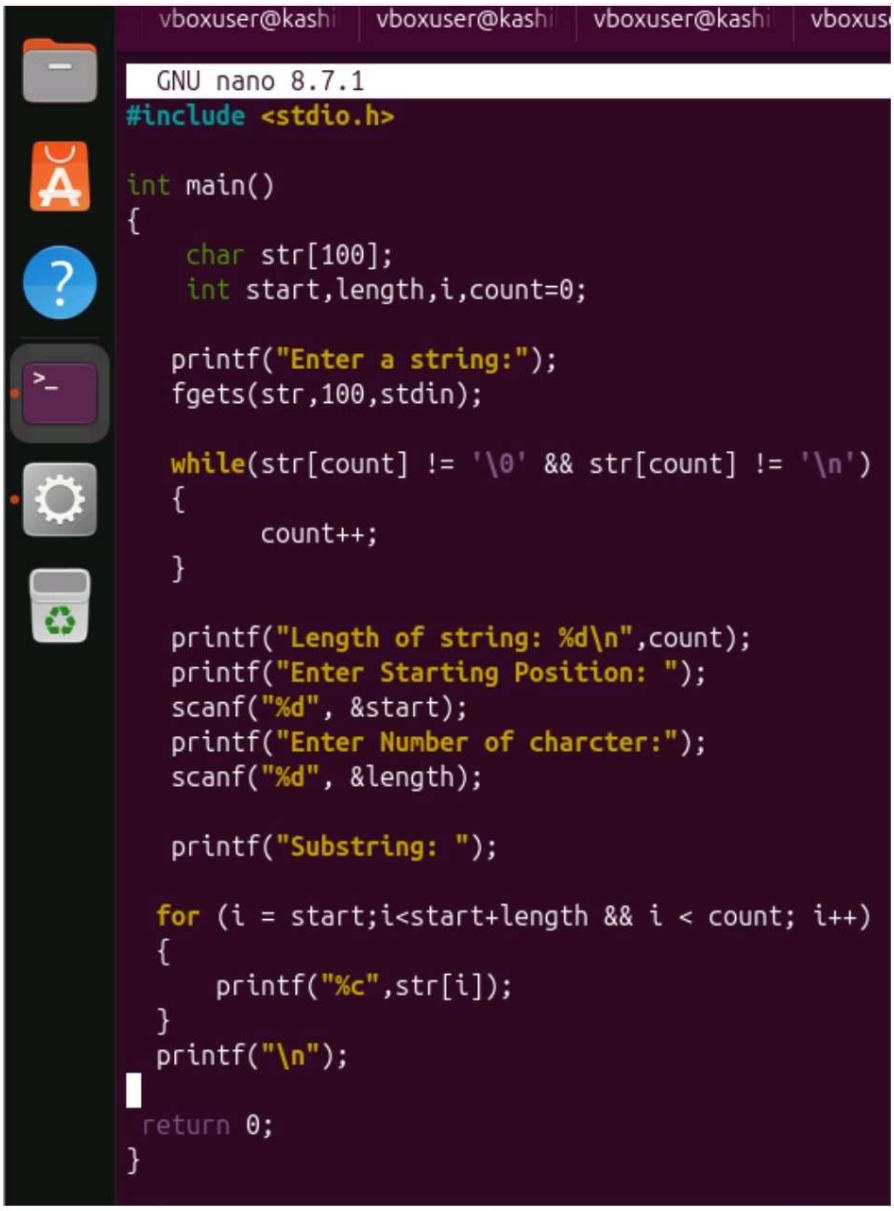
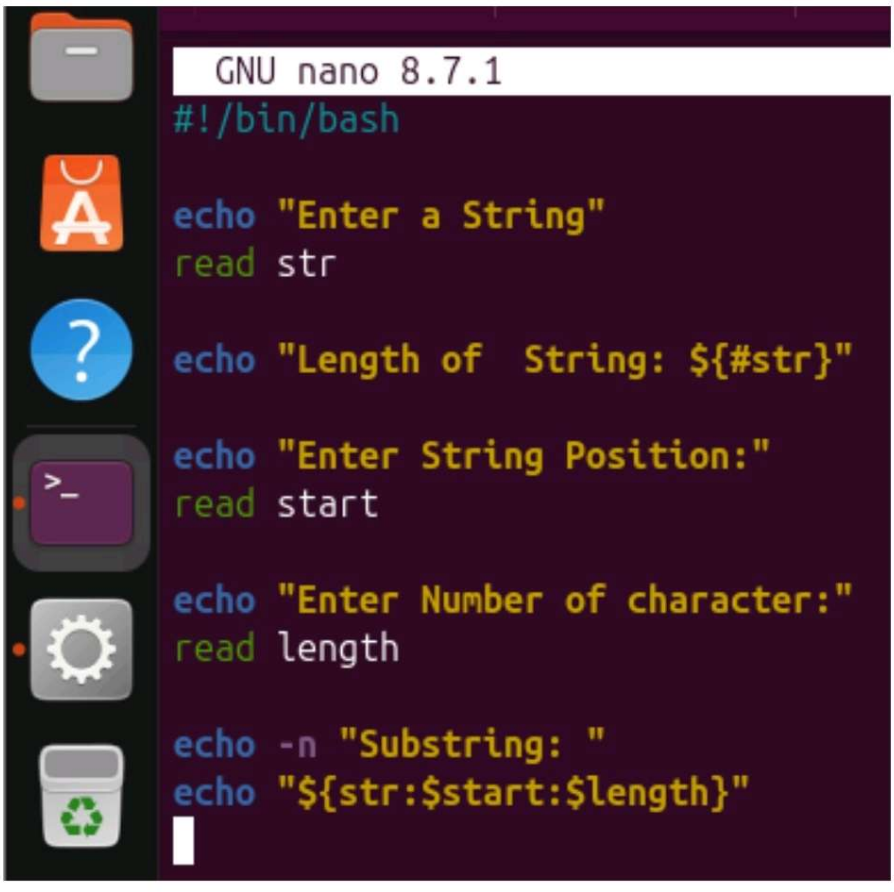
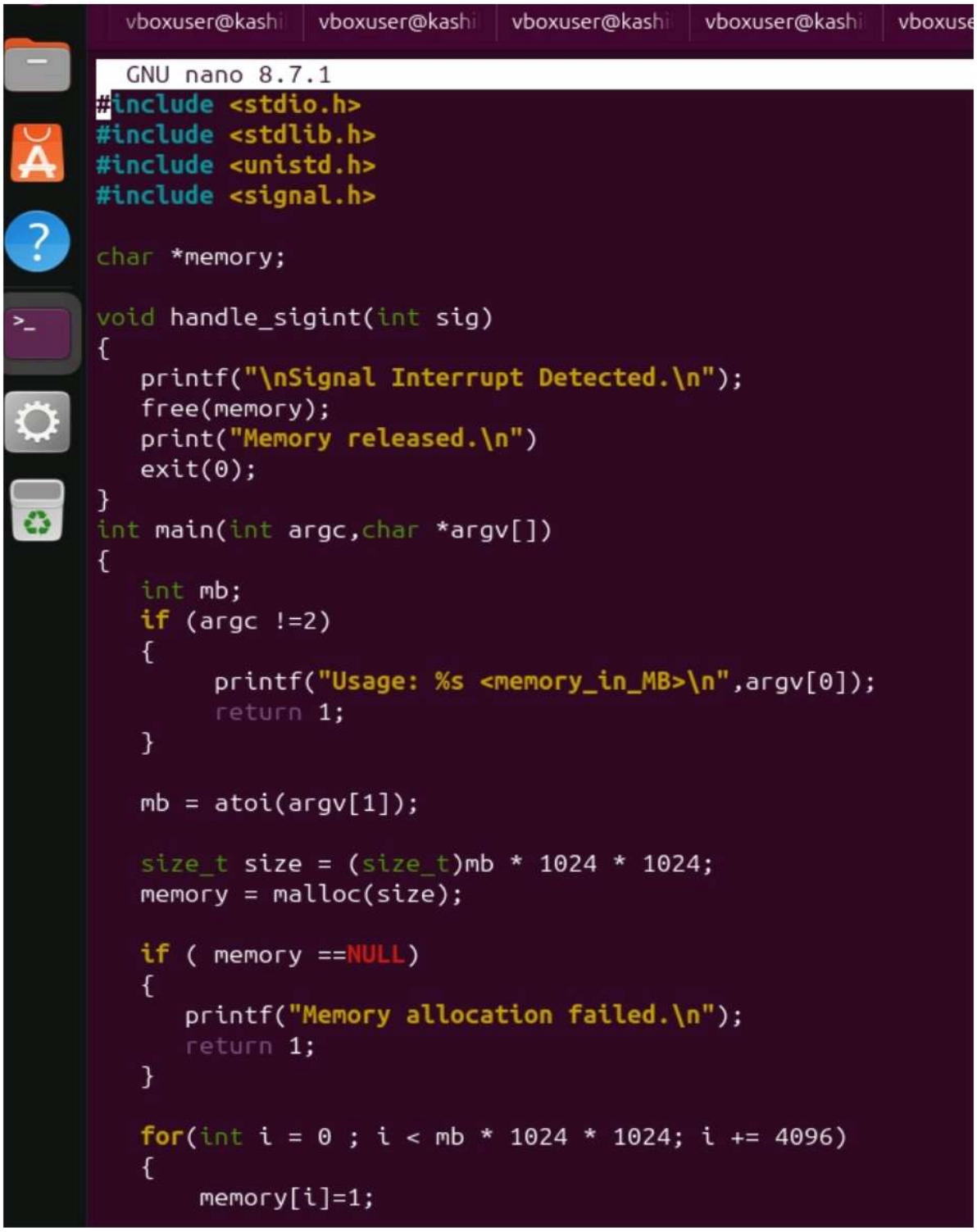

vboxuser@kashi
vboxuser@kashi
vboxuser@kashi
vbox
vboxuser@kashik:~$ nano exp5.sh
vboxuser@kashik:~$ chmod +x exp5.sh
vboxuser@kashik:~$ ./exp5.sh
Enter a String
Kashik Balla
Length of String: 12
Enter String Position:
3
Enter Number of character:
5
Substring: hik B
vboxuser@kashik:~$ gcc exp5.c -o string
vboxuser@kashik:~$ ./string
Enter a string:CRISTIANO RONALDO
Length of string: 17
Enter Starting Position: 7
Enter Number of charcter:8
Substring: NO RONAL
vboxuser@kashik:~$ time ./string
Enter a string:Kashik Balla
Length of string: 12
Enter Starting Position: 3
Enter Number of charcter:5
Substring: hik B
real 0m13.404s
user 0m0.001s
sys 0m0.001s
vboxuser@kashik:~$ time ./exp5.sh
Enter a String
Kashik Balla
Length of String: 12
Enter String Position:
3
Enter Number of character:
5
Substring: hik B
real 0m25.625s
user 0m0.004s
sys 0m0.001s
vboxuser@kashik:~$


vboxuser@kashi
vboxuser@kashi
vboxuser@kashi
GNU nano 8.7.1
#include <stdio.h>
int main()
{
char str[100];
int start,length,i,count=0;
printf("Enter a string:");
fgets(str,100,stdin);
while(str[count] != '\0' && str[count] != '\n')
{
count++;
}
printf("Length of string: %d\n",count);
printf("Enter Starting Position: ");
scanf("%d", &start);
printf("Enter Number of charcter:");
scanf("%d", &length);
printf("Substring: ");
for (i = start;i<start+length && i < count; i++)
{
printf("%c",str[i]);
}
printf("\n");
return 0;
}


GNU nano 8.7.1
#!/bin/bash
echo "Enter a String"
read str
echo "Length of String: ${#str}"
echo "Enter String Position:"
read start
echo "Enter Number of character:"
read length
echo -n "Substring: "
echo "${str:$start:$length}"


vboxuser@kashi
vboxuser@kashi
vboxuser@kashi
vboxuser@kashi
vboxuse
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
char *memory;
void handle_sigint(int sig)
{
printf("\nSignal Interrupt Detected.\n");
free(memory);
print("Memory released.\n")
exit(0);
}
int main(int argc,char *argv[])
{
int mb;
if (argc !=2)
{
printf("Usage: %s <memory_in_MB>\n",argv[0]);
return 1;
}
mb = atoi(argv[1]);
size_t size = (size_t)mb * 1024 * 1024;
memory = malloc(size);
if (memory ==NULL)
{
printf("Memory allocation failed.\n");
return 1;
}
for(int i = 0 ; i < mb * 1024 * 1024; i += 4096)
{
memory[i]=1;